ROTATING COMPASS DIAL VISUALIZATION USING MIDPOINT CIRCLE AND DDA LINE DRAWING ALGORITHMS

Mini Project Report

Submitted by,

Neha Miriam Jose

Roll No :14

S6,CSE-C

Course: Computer Graphics and Image Processing (23CST304)

CONTENTS

SECTION	PAGE
Abstract	2
Introduction	3
Methodology	4
Source Code	5
Result	14
Conclusion	16

ABSTRACT

This project presents an animated rotating compass dial that demonstrates the practical application of fundamental computer graphics algorithms. The system uses two important rasterization techniques: the Midpoint Circle Algorithm (Bresenham's Circle Algorithm) and the Digital Differential Analyzer (DDA) Line Drawing Algorithm, which are widely used for drawing geometric shapes in computer graphics.

The compass dial contains a rotating needle that smoothly moves through all 360 degrees. During rotation, the needle briefly pauses at important directional points such as the cardinal directions (North, East, South, West) and the intercardinal directions (Northeast, Southeast, Southwest, Northwest), which occur at every 45 degrees. These pauses make the visualization easier to understand and provide clear reference points for compass navigation. The fixed background dial displays direction labels and degree markers to enhance readability.

The application is implemented in Java using the Swing GUI framework and runs as a standalone program without external dependencies. It achieves smooth animation while using simple integer-based calculations, demonstrating the efficiency of the implemented algorithms

This project demonstrates how fundamental computer graphics algorithms can be applied to create an interactive and visually appealing rotating compass dial. By implementing rasterization algorithms from first principles, it shows that efficient and accurate graphics rendering can be achieved through proper algorithm design and implementation. The application serves as a useful educational tool for understanding basic computer graphics concepts while also demonstrating how these algorithms can be used to build practical visualization systems.

INTRODUCTION

Computer graphics is a fundamental area of computer science that deals with the creation and manipulation of visual content using computers. It plays an important role in many modern applications such as simulations, games, visualization systems, and user interfaces. Behind these graphical systems are several basic algorithms that help in efficiently drawing shapes and lines on a digital display. Understanding these algorithms is essential for learning the core principles of computer graphics.

This project focuses on the development of a rotating compass dial visualization that demonstrates the implementation of two important computer graphics algorithms: the Midpoint Circle Algorithm and the Digital Differential Analyzer (DDA) Line Drawing Algorithm. These algorithms are commonly used to generate circles and straight lines on raster displays using simple mathematical calculations.

METHODOLOGY

1. System Design

The first step involved designing the overall structure of the application. The compass dial was planned to include a circular background with directional labels and a rotating needle. The dial remains fixed while the needle rotates through 360 degrees to simulate the behavior of a real compass.

2. Implementation of the Midpoint Circle Algorithm

The Midpoint Circle Algorithm was used to draw the circular boundary of the compass dial. This algorithm efficiently generates points of a circle using integer calculations. By plotting symmetrical points around the center, the circular dial was created accurately and efficiently.

3. Implementation of the DDA Line Drawing Algorithm

The DDA (Digital Differential Analyzer) Line Drawing Algorithm was used to draw straight lines, including the compass needle and direction markers. The algorithm calculates intermediate points between the start and end coordinates to produce smooth and consistent lines.

4. Dial Markings and Direction Labels

Directional labels such as North, East, South, and West, along with intercardinal directions, were added to the dial. Degree markers were also included around the circle to represent the full 360-degree rotation.

5. Needle Rotation and Animation

A rotation mechanism was implemented to animate the compass needle. The needle rotates continuously through all angles from 0° to 360°. Short pauses were introduced at major directional points to make the animation clearer and easier to observe.

6. Graphical User Interface Development

The application interface was developed using the Java Swing framework. Swing components were used to create the display window, handle rendering, and manage the animation updates.

SOURCE CODE

```java
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.util.ArrayList;
import java.util.List;

/**
 * Rotating Compass Dial Visualization
 * Using Midpoint Circle Algorithm and DDA Line Drawing Algorithm
 * All in one file - Single application for CGIP Animation project
 */
public class CompassDial extends JFrame {
    public CompassDial() {
        setTitle("Rotating Compass Dial - CGIP Animation");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setResizable(false);

        CompassPanel panel = new CompassPanel();
        add(panel);

        setSize(800, 800);
        setLocationRelativeTo(null);
        setVisible(true);
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new CompassDial());
    }
}

/**
 * Panel for drawing the rotating compass dial
 */
class CompassPanel extends JPanel {
    private double rotationAngle = 0;
    private final int centerX = 400;
```

```java
    private final int centerX = 400;
    private final int centerY = 400;
    private final int radius = 150;
    private final int tickRadius = 140;

    private MidpointCircle circleAlgorithm;
    private DDALine ddaLine;

    public CompassPanel() {
        circleAlgorithm = new MidpointCircle();
        ddaLine = new DDALine();

        // Animation timer
        Timer animationTimer = new Timer(30, new ActionListener() {
            private int pauseCounter = 0;
            @Override
            public void actionPerformed(ActionEvent e) {
                // Check if at 45-degree interval for pause
                int currentPos = (int) rotationAngle;
                if (currentPos % 45 == 0 && pauseCounter < 10) {
                    pauseCounter++; // Pause for 10 frames at 45-degree intervals
                } else {
                    pauseCounter = 0;
                    rotationAngle -= 2; // Rotate by 2 degrees each frame (clockwise)
                }

                if (rotationAngle <= 0) {
                    rotationAngle = 360;
                }
                repaint();
            }
        });
        animationTimer.start();
    }

    @Override
```

```java
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        Graphics2D g2d = (Graphics2D) g;
        g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);

        // Draw gradient background
        GradientPaint bgGradient = new GradientPaint(
            0, 0, new Color(25, 45, 85),
            getWidth(), getHeight(), new Color(15, 25, 50)
        );
        g2d.setPaint(bgGradient);
        g2d.fillRect(0, 0, getWidth(), getHeight());

        // Draw outer circle using midpoint circle algorithm
        g2d.setColor(new Color(255, 215, 0));
        g2d.setStroke(new BasicStroke(4));
        drawCircleWithMidpoint(g2d, centerX, centerY, radius);
```

```java
    // Draw second outer circle
    g2d.setColor(new Color(0, 191, 255));
    g2d.setStroke(new BasicStroke(2));
    drawCircleWithMidpoint(g2d, centerX, centerY, radius + 10);

    // Draw inner circle
    g2d.setColor(new Color(0, 255, 127));
    g2d.setStroke(new BasicStroke(3));
    drawCircleWithMidpoint(g2d, centerX, centerY, radius - 20);

    // Draw center decorative circle
    g2d.setColor(new Color(255, 0, 127));
    g2d.setStroke(new BasicStroke(2));
    drawCircleWithMidpoint(g2d, centerX, centerY, 30);

    // Draw compass rose with rotating ticks
    drawCompassTicks(g2d);

    // Draw cardinal directions
    drawCardinalDirections(g2d);

    // Draw center point
    g2d.setColor(Color.YELLOW);
    g2d.fillOval(centerX - 7, centerY - 7, 14, 14);
    g2d.setColor(new Color(255, 0, 127));
    g2d.fillOval(centerX - 4, centerY - 4, 8, 8);

    // Draw rotation indicator line
    drawRotationIndicator(g2d);

    // Draw info text
    drawInfo(g2d);
}

private void drawCircleWithMidpoint(Graphics2D g2d, int cx, int cy, int r) {
```

```java
private void drawCircleWithMidpoint(Graphics2D g2d, int cx, int cy, int r) {
    int[][] points = circleAlgorithm.getCirclePoints(cx, cy, r);

    for (int[] point : points) {
        g2d.drawLine(point[0], point[1], point[0], point[1]);
    }
}

private void drawCompassTicks(Graphics2D g2d) {
    // Draw 360 ticks (one for each degree)
    for (int degree = 0; degree < 360; degree += 1) {
        double angle = Math.toRadians(degree); // Fixed position - no rotation

        int length = (degree % 10 == 0) ? 15 : 8;
        Color tickColor;
```

```java
        if (degree % 90 == 0) {
            // Cardinal directions - bright colors
            length = 25;
            g2d.setStroke(new BasicStroke(4));
            if (degree == 0) {
                tickColor = new Color(255, 0, 0);      // Red - North
            } else if (degree == 90) {
                tickColor = new Color(0, 255, 0);      // Green - East
            } else if (degree == 180) {
                tickColor = new Color(0, 0, 255);      // Blue - South
            } else {
                tickColor = new Color(255, 255, 0);    // Yellow - West
            }
        } else if (degree % 45 == 0) {
            // Intercardinal directions
            length = 18;
            g2d.setStroke(new BasicStroke(3));
            tickColor = new Color(255, 165, 0);        // Orange
        } else if (degree % 10 == 0) {
            g2d.setStroke(new BasicStroke(2));
            tickColor = new Color(0, 255, 255);        // Cyan
        } else {
            g2d.setStroke(new BasicStroke(1));
            tickColor = new Color(200, 200, 200);      // Light gray
        }

        g2d.setColor(tickColor);

        // Calculate tick positions
        int x1 = centerX + (int)(Math.cos(angle) * (tickRadius));
        int y1 = centerY + (int)(Math.sin(angle) * (tickRadius));
        int x2 = centerX + (int)(Math.cos(angle) * (tickRadius - length));
        int y2 = centerY + (int)(Math.sin(angle) * (tickRadius - length));
```

```java
            // Draw line using DDA algorithm
            drawLineWithDDA(g2d, x1, y1, x2, y2);
        }
    }

    private void drawCardinalDirections(Graphics2D g2d) {
        g2d.setFont(new Font("Arial", Font.BOLD, 28));

        String[] directions = {"N", "E", "S", "W"};
        String[] fullNames = {"North", "East", "South", "West"};
        int[] angles = {0, 90, 180, 270};
        Color[] colors = {
            new Color(255, 0, 0),      // Red - North
            new Color(0, 255, 0),      // Green - East
            new Color(0, 0, 255),      // Blue - South
            new Color(255, 255, 0)     // Yellow - West
        };
        int textRadius = 100;
```

```java
for (int i = 0; i < directions.length; i++) {
    double angle = Math.toRadians(angles[i]); // Fixed position - no rotation
    int x = centerX + (int)(Math.cos(angle) * textRadius);
    int y = centerY + (int)(Math.sin(angle) * textRadius);

    FontMetrics fm = g2d.getFontMetrics();
    String text = directions[i];

    // Draw shadow/outline effect
    g2d.setColor(new Color(0, 0, 0, 200));
    for (int dx = -2; dx <= 2; dx++) {
        for (int dy = -2; dy <= 2; dy++) {
            if (dx != 0 || dy != 0) {
                g2d.drawString(text,
                               x - fm.stringWidth(text) / 2 + dx,
                               y + fm.getAscent() / 2 + dy);
            }
        }
    }

    // Draw main text in bright color
    g2d.setColor(colors[i]);
    g2d.drawString(text,
                   x - fm.stringWidth(text) / 2,
                   y + fm.getAscent() / 2);

    // Draw full direction name below cardinal letters
    g2d.setFont(new Font("Arial", Font.PLAIN, 12));
    g2d.setColor(colors[i]);
    FontMetrics fmSmall = g2d.getFontMetrics();
    g2d.drawString(fullNames[i],
                   x - fmSmall.stringWidth(fullNames[i]) / 2,
                   y + fm.getAscent() + 15);
```

```java
        g2d.setFont(new Font("Arial", Font.BOLD, 28));
    }
}

private void drawRotationIndicator(Graphics2D g2d) {
    g2d.setColor(new Color(255, 50, 150));
    g2d.setStroke(new BasicStroke(4));

    double angle = Math.toRadians(-rotationAngle);
    int x2 = centerX + (int)(Math.cos(angle) * (radius - 30));
    int y2 = centerY + (int)(Math.sin(angle) * (radius - 30));

    // Draw line using DDA
    drawLineWithDDA(g2d, centerX, centerY, x2, y2);

    // Draw arrow head
    drawArrowHead(g2d, x2, y2, angle);

    // Draw glow effect
    g2d.setColor(new Color(255, 100, 200, 100));
    g2d.setStroke(new BasicStroke(6));
    drawLineWithDDA(g2d, centerX, centerY, x2, y2);
}
```

```java
    private void drawArrowHead(Graphics2D g2d, int x, int y, double angle) {
        int arrowSize = 15;
        int[] arrowX = new int[3];
        int[] arrowY = new int[3];

        arrowX[0] = x;
        arrowY[0] = y;

        arrowX[1] = (int)(x - arrowSize * Math.cos(angle - Math.PI / 6));
        arrowY[1] = (int)(y - arrowSize * Math.sin(angle - Math.PI / 6));

        arrowX[2] = (int)(x - arrowSize * Math.cos(angle + Math.PI / 6));
        arrowY[2] = (int)(y - arrowSize * Math.sin(angle + Math.PI / 6));

        g2d.setColor(new Color(255, 50, 150));
        g2d.fillPolygon(arrowX, arrowY, 3);

        g2d.setColor(new Color(255, 200, 100));
        g2d.setStroke(new BasicStroke(2));
        g2d.drawPolygon(arrowX, arrowY, 3);
    }

    private void drawLineWithDDA(Graphics2D g2d, int x1, int y1, int x2, int y2) {
        int[][] points = ddaLine.getLinePoints(x1, y1, x2, y2);

        for (int[] point : points) {
            g2d.drawLine(point[0], point[1], point[0], point[1]);
        }
    }

    private void drawInfo(Graphics2D g2d) {
        g2d.setFont(new Font("Arial", Font.BOLD, 16));

        String info = String.format("Rotation Angle: %.1f°", rotationAngle);
```

```java
        // Draw background for text
        FontMetrics fm = g2d.getFontMetrics();
        int textWidth = fm.stringWidth(info);
        g2d.setColor(new Color(0, 0, 0, 150));
        g2d.fillRect(15, 15, textWidth + 20, 35);

        // Draw text in bright cyan
        g2d.setColor(new Color(0, 255, 255));
        g2d.drawString(info, 25, 42);

        // Draw algorithm info
        String algoInfo = "Algorithms: Midpoint Circle & DDA Line";
        g2d.setColor(new Color(0, 0, 0, 150));
        int algoWidth = fm.stringWidth(algoInfo);
        g2d.fillRect(15, 60, algoWidth + 20, 35);

        g2d.setColor(new Color(0, 255, 127));
        g2d.drawString(algoInfo, 25, 87);
    }
}

/**
 * Midpoint Circle Algorithm (Bresenham's Circle Algorithm)
 * Used for drawing circles with integer coordinates
 */
```

```java
    public int[][] getCirclePoints(int centerX, int centerY, int radius) {
        List<int[]> points = new ArrayList<>();

        int x = 0;
        int y = radius;
        int d = 3 - 2 * radius; // Initial decision parameter

        while (x <= y) {
            // Add all 8 octants
            addSymmetricPoints(points, centerX, centerY, x, y);

            // Update decision parameter and coordinates
            if (d < 0) {
                d = d + 4 * x + 6;
            } else {
                d = d + 4 * (x - y) + 10;
                y--;
            }
            x++;
        }

        return points.toArray(new int[0][]);
    }

    /**
     * Add 8 symmetric points of the circle
     */
    private void addSymmetricPoints(List<int[]> points, int centerX, int centerY, int x, int y) {
        // All 8 octants
        points.add(new int[]{centerX + x, centerY + y});
        points.add(new int[]{centerX - x, centerY + y});
        points.add(new int[]{centerX + x, centerY - y});
        points.add(new int[]{centerX - x, centerY - y});
        points.add(new int[]{centerX + y, centerY + x});
```

```java
        points.add(new int[]{centerX - y, centerY + x});
        points.add(new int[]{centerX + y, centerY - x});
        points.add(new int[]{centerX - y, centerY - x});
    }

    /**
     * Get circle points within a specific angle range (for drawing arc segments)
     */
    public int[][] getCircleArc(int centerX, int centerY, int radius, double startAngle, double endAngle)
        List<int[]> points = new ArrayList<>();

        int x = 0;
        int y = radius;
        int d = 3 - 2 * radius;

        while (x <= y) {
            addArcSymmetricPoints(points, centerX, centerY, x, y, startAngle, endAngle);

            if (d < 0) {
                d = d + 4 * x + 6;
            } else {
                d = d + 4 * (x - y) + 10;
                y--;
            }
            x++;
        }

        return points.toArray(new int[0][]);
    }
```

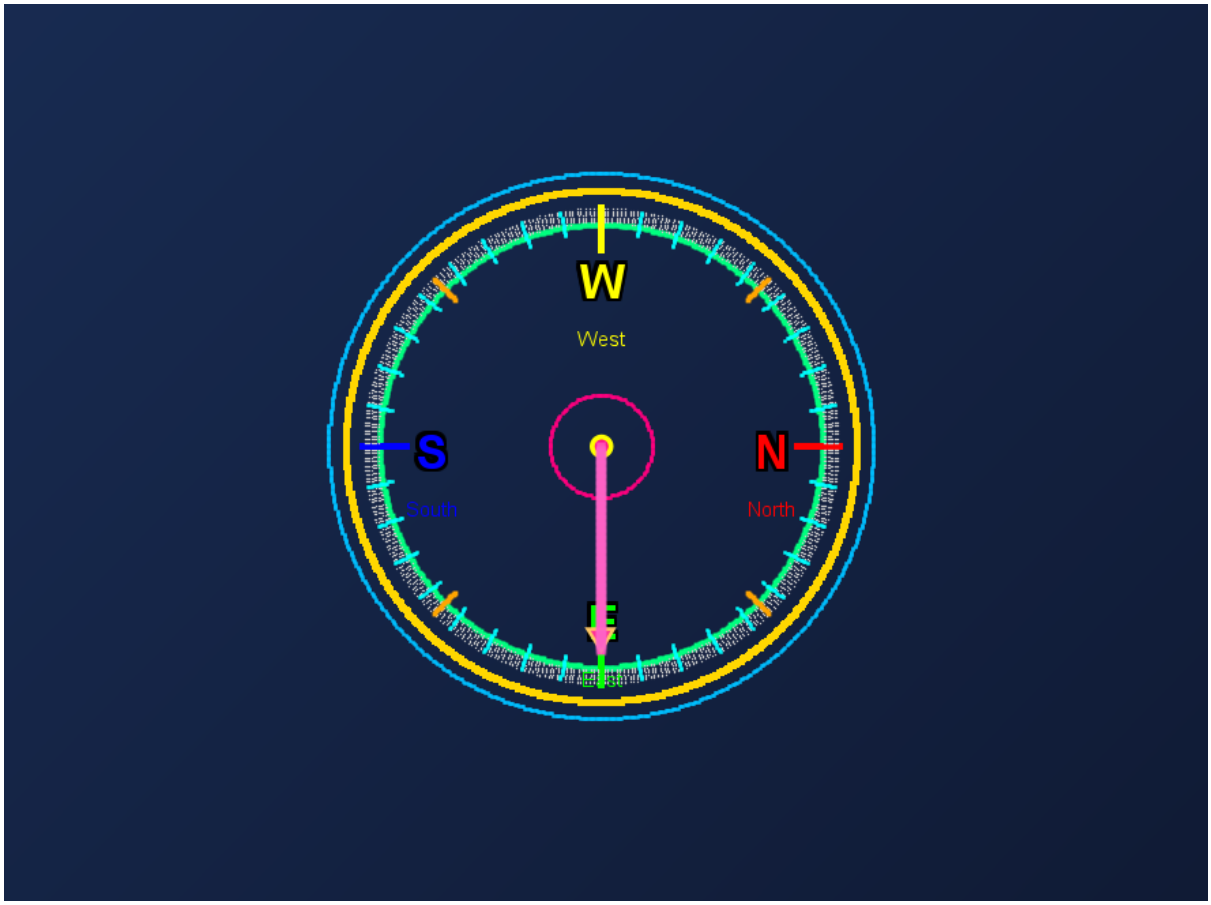
```java
    private void addArcSymmetricPoints(List<int[]> points, int centerX, int centerY,
                                       int x, int y, double startAngle, double endAngle) {
        int[][] octantPoints = {
            {centerX + x, centerY + y},
            {centerX - x, centerY + y},
            {centerX + x, centerY - y},
            {centerX - x, centerY - y},
            {centerX + y, centerY + x},
            {centerX - y, centerY + x},
            {centerX + y, centerY - x},
            {centerX - y, centerY - x}
        };

        for (int[] point : octantPoints) {
            double angle = Math.atan2(point[1] - centerY, point[0] - centerX);
            if (isAngleInRange(angle, startAngle, endAngle)) {
                points.add(point);
            }
        }
    }

    /**
     * Check if angle is within the given range
     */
    private boolean isAngleInRange(double angle, double startAngle, double endAngle) {
        while (angle < startAngle) angle += 2 * Math.PI;
        while (angle > endAngle) angle -= 2 * Math.PI;
        return angle >= startAngle && angle <= endAngle;
    }
```

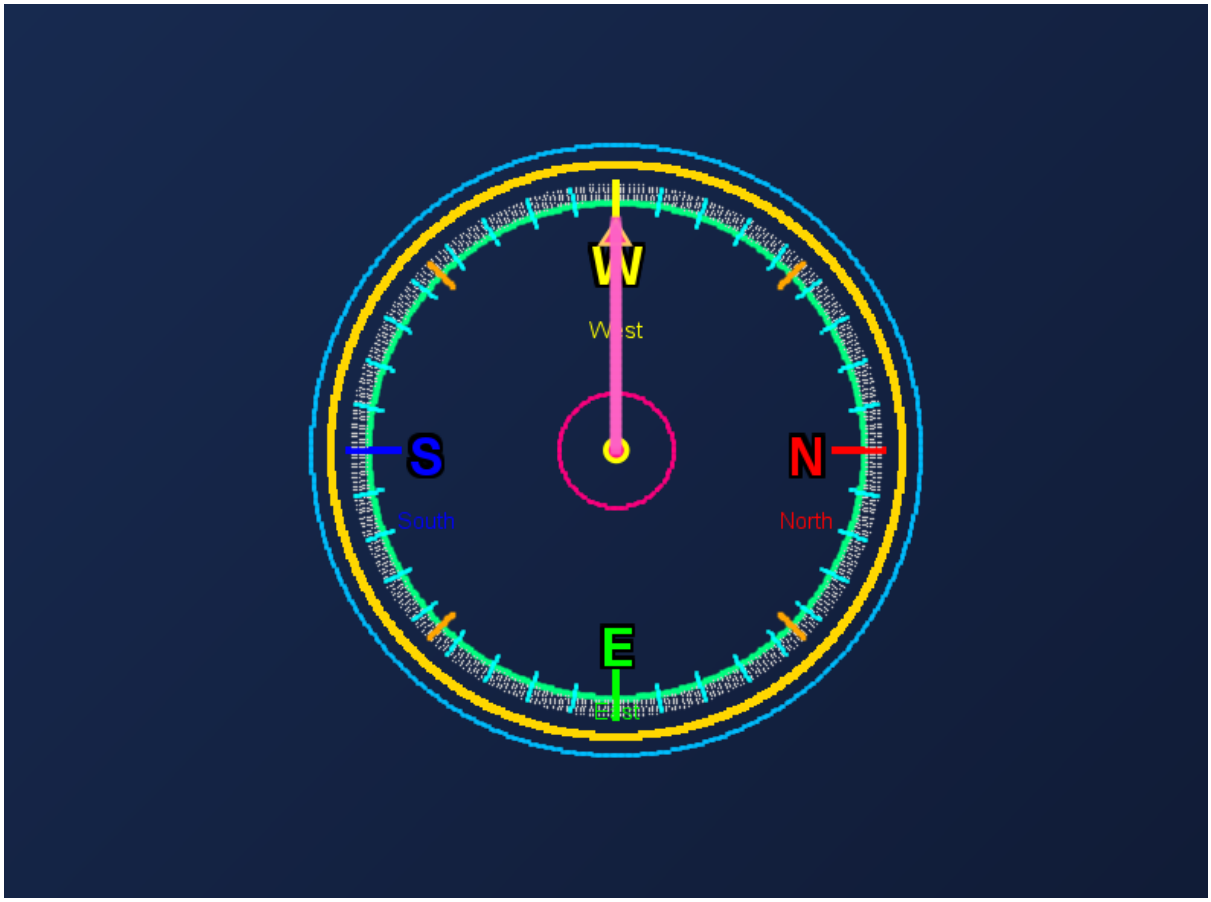
```java
class DDALine {

    /**
     * Get all points on a line using the DDA algorithm
     * @param x1 starting X coordinate
     * @param y1 starting Y coordinate
     * @param x2 ending X coordinate
     * @param y2 ending Y coordinate
     * @return array of [x, y] coordinates
     */
    public int[][] getLinePoints(int x1, int y1, int x2, int y2) {
        List<int[]> points = new ArrayList<>();

        int dx = x2 - x1;
        int dy = y2 - y1;

        int steps = Math.max(Math.abs(dx), Math.abs(dy));

        if (steps == 0) {
            points.add(new int[]{x1, y1});
            return points.toArray(new int[0][]);
        }

        float xIncrement = (float) dx / steps;
        float yIncrement = (float) dy / steps;

        float x = x1;
        float y = y1;

        for (int i = 0; i <= steps; i++) {
            points.add(new int[]{Math.round(x), Math.round(y)});
            x += xIncrement;
            y += yIncrement;
        }
```

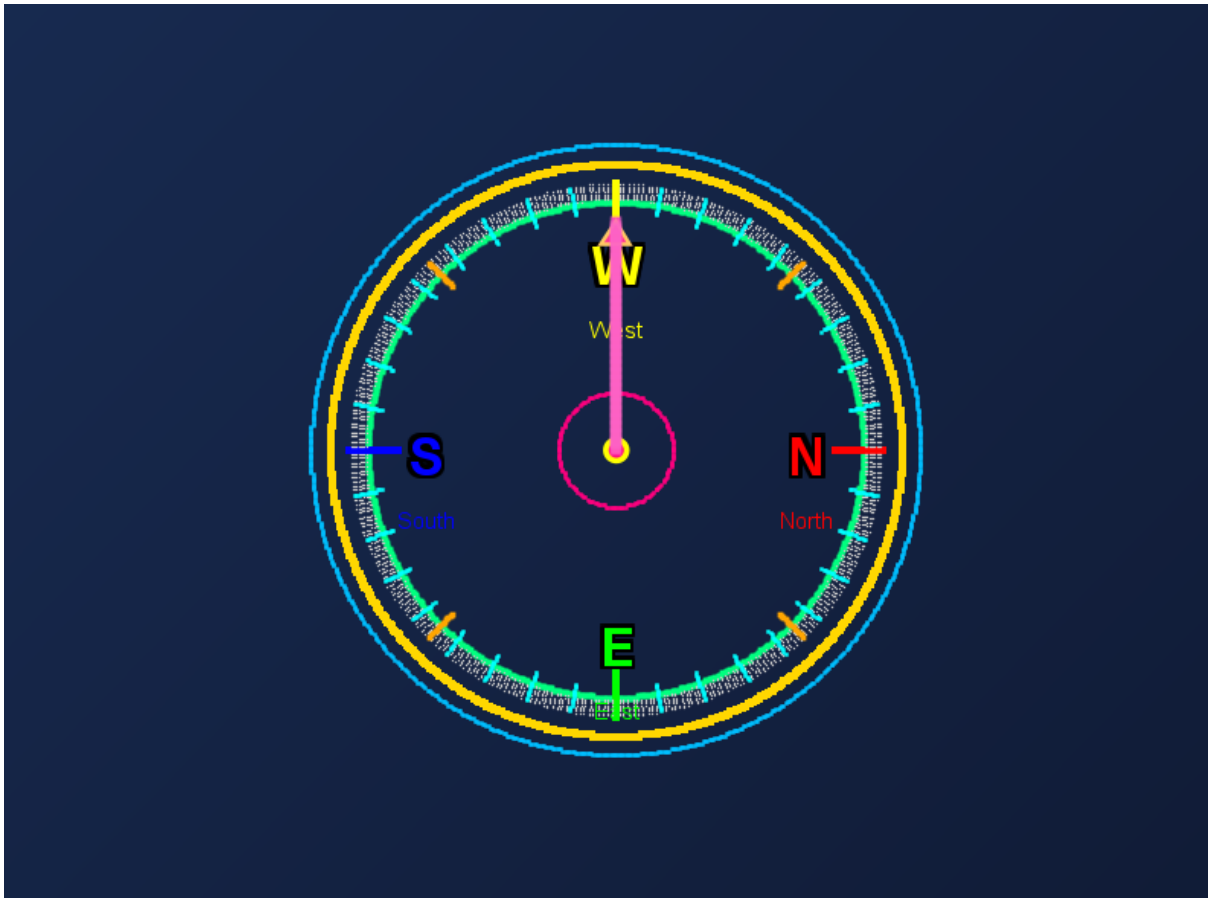
```java
        return points.toArray(new int[0][]);
    }

    /**
     * Alternative DDA with Bresenham-like approach for integer arithmetic
     */
    public int[][] getLinePointsInteger(int x1, int y1, int x2, int y2) {
        List<int[]> points = new ArrayList<>();

        int dx = Math.abs(x2 - x1);
        int dy = Math.abs(y2 - y1);
        int sx = (x1 < x2) ? 1 : -1;
        int sy = (y1 < y2) ? 1 : -1;

        int x = x1;
        int y = y1;

        if (dx > dy) {
            int err = dx / 2;
            while (x != x2) {
                points.add(new int[]{x, y});
                err -= dy;
                if (err < 0) {
                    y += sy;
                    err += dx;
                }
                x += sx;
            }
        } else {
            int err = dy / 2;
            while (y != y2) {
                points.add(new int[]{x, y});
                err -= dx;
                if (err < 0) {
```

```java
                    x += sx;
                    err += dy;
                }
                y += sy;
            }
        }

        points.add(new int[]{x2, y2});
        return points.toArray(new int[0][]);
    }
}
```

RESULT




W
West
S
South
N
North
E


W
West
S
South
N
North
E

CONCLUSION

This project successfully demonstrates the practical implementation of fundamental computer graphics algorithms through the development of a rotating compass dial visualization. By applying the Midpoint Circle Algorithm and the DDA Line Drawing Algorithm, the project shows how basic rasterization techniques can be used to generate graphical elements such as circles and straight lines efficiently.

The application displays a smoothly rotating compass needle along with clearly marked directional indicators, providing an interactive and visually understandable representation of compass movement. The use of Java and the Swing framework allowed the system to be developed as a standalone application with smooth animation and reliable performance.

Overall, this project highlights the importance of understanding core computer graphics algorithms and their role in graphical rendering. It also demonstrates how these fundamental concepts can be used to create simple yet effective visualizations for educational and practical purposes.